

Module 4 — Subagents

Lesson 4.4

Parallel orchestration

Spawn multiple subagents in a single message. Their work runs concurrently; you collect the results and synthesize.

Mastering Claude Code — An E-Course for Power Users

Why it matters

For genuinely independent units of work, parallel agents cut wall-clock time roughly N-fold (for N agents). The trick is recognizing which work is actually independent and writing briefs that don't accidentally couple them.

The mechanics

Inside one assistant turn, issue multiple Agent tool calls in the same block:

```
[single message with three tool calls]:
  Agent(description="audit api/", subagent_type="Explore", prompt="...")
  Agent(description="audit web/", subagent_type="Explore", prompt="...")
  Agent(description="audit jobs/", subagent_type="Explore", prompt="...")
```

All three boot, run, and return concurrently. Your next turn aggregates the three reports.

The harness handles the parallelism. You just have to issue the calls together; don't await one and then call the next.

What's actually parallelizable

The honest test: would the briefs change if you ran them in any order?

Parallel-safe:

- Audit each of three directories for the same issue
- Generate a changelog entry, a release-notes entry, and a tweet from the same diff
- Run a Python migration suggestion, a Ruby migration suggestion, and a Go migration suggestion for the same schema change (then compare)
- Look up "what does X mean in our codebase" three different ways (semantic search, naming search, doc search)

Not parallel-safe:

- Step 1 produces a list of files; step 2 acts on each file. Step 2's brief depends on step 1's output.
- Implementations that touch overlapping code (they'll conflict on merge)
- Anything where one agent's findings are inputs to another's brief

The fan-out pattern in detail

For N independent units of identical work:

```
Briefs differ only in scope:
  Agent 1: Brief("...for src/billing/...")
  Agent 2: Brief("...for src/auth/...")
  Agent 3: Brief("...for src/users/...")
```

Shared template:

```
"For the given dir, find every place we still use the deprecated
`getUserSync()`. Report file:line list. Under 100 words."
```

Each agent works in its own context. When all return, you have three reports to synthesize into a single picture.

The compare-and-contrast pattern

For tasks where multiple approaches each warrant a fair look:

Agent 1: Plan the migration as a synchronous backfill.
Agent 2: Plan the migration as an async-job backfill.
Agent 3: Plan the migration using dual-write + cutover.

Then YOU read all three plans and pick (or synthesize).

This gives you genuine alternatives instead of the first thing that came to mind.

The cross-perspective pattern

For tasks where independence is the point:

Agent 1: Review the diff. Independent perspective, no exposure to my reasoning.
Agent 2 (security-focused): Same diff, look for security issues only.
Agent 3 (perf-focused): Same diff, look for performance regressions only.

Specialization plus parallelism. Each agent stays focused.

Limits

Limit	Practical advice
Concurrent agents	Cap at 3 in one message — beyond that you'll see degraded latency and rate limits
Recursion	Agents typically can't spawn their own agents — don't design hierarchies
Shared state	Agents don't see each other. If they need shared facts, put them in CLAUDE.md or in each brief
Cost	N agents = roughly N times the spend. Trade is wall-clock for budget.

Synthesizing the results

After parallel fan-out, your next move matters as much as the spawn:

"All three agents reported. Aggregate their file:line findings. Deduplicate. Rank by suspected severity. Give me the top 5."

That synthesis is the user-side work the parallelism enables. Don't just dump three reports into the chat — do the aggregation.

When parallel is the wrong choice

- The work is too small. Three agents at 30 seconds each is 30 seconds wall-clock for what one agent could've done in 60. Margin is thin; coordination cost might outweigh.
- You'd rather see incremental progress and adjust. Parallel commits you to a fixed plan; sequential lets you pivot after each step.
- Cost matters more than time. Sequential = cheaper.

Try it

Pick a real audit task in your repo (e.g., "find every place we hardcode a timezone"):

- 1 Do it inline. Note the time.
- 2 Do it with one subagent. Note the time.
- 3 Do it with three subagents, one per top-level subdirectory. Note the time and the context cost in your main session.

Pick the workflow that matches your typical situation. Don't always reach for parallelism; reach for it when it pays.

Gotchas

- **Don't run parallel agents that write to the same files.** They'll race and corrupt. Use worktree isolation if write conflicts are possible.
- **Agents don't see each other's results.** If one needs another's output, the workflow is sequential, not parallel.
- **Three reports = three contexts to read.** Budget your reading time accordingly. Cap each brief's word count.
- **Rate limits are real.** Bursting 5 agents at once may queue or fail. Stay within ~3 concurrent.

Further reading

- 4.5 Worktree isolation — when parallel agents need to write safely
- 2.6 When to delegate — when to even consider parallel

Next

→ 4.5 Worktree isolation